Full-Stack Take-Home Exercise

Mobile App Home Screen Editor

Objective

Build a **full-stack web application** that lets users preview and modify an example mobile app home screen in real time. Users configure three types of elements: an image carousel, a text section, and a call-to-action (CTA) button. The preview updates live as they edit.

The configuration must be persisted to a backend configuration service via a **private API**. Reloading the page should restore the saved configuration.

This exercise evaluates frontend fundamentals (component design, state management, UX, input handling, and code quality) along with backend/API design, server-side data access, and overall architectural decision-making. We're most interested in a functional, well-structured foundation, so favour clean and extensible over broad and rushed.

Technology Requirements

- Framework: **React Router v7 in framework mode** or **Next.js** with a server runtime.
- Language: TypeScript (preferred) or JavaScript.
- API Protocol: REST or GraphQL.
- State management: React Context and Hooks, or an equivalent idiomatic approach.

Functional Requirements

Preview Screen Components

- Carousel Section:
    - Should scroll horizontally, you can use a third-party plugin for the carousel features.
    - Allow adding, editing, and removing image URLs from the carousel.
    - Provide options to display the carousel in portrait, landscape, or square view.

- Textarea Section:
    - Display a title and a description.
    - Allow changing the text of the title and description.
    - Allow changing the hex color value for the title and description.

- Call to Action Section:

    - Display a button with a label.
    - Allow changing the button label and link.
    - Allow changing the hex color value of the button and the label text.

Editor Behaviour

- Changes should be reflected in real-time in the preview.
- Provide a user interface to input and modify the required fields for each section.
- Allow the user to add multiple sections of each type to the screen (e.g. more than one carousel), and reorder sections on the screen.
- Ensure the UI is responsive and user-friendly.

Import / Export

- Support exporting the current configuration as a JSON file and importing a valid configuration file to restore a previous state.
- Handle invalid or malformed files gracefully.

Backend & Persistence Requirements

Configuration Service

Implement a backend service responsible for storing and retrieving app home screen configurations.

Each configuration should include, at minimum:

- id (string)
- schemaVersion (number)
- updatedAt (timestamp)
- data (configuration payload)

The configuration payload should capture the ordered list of sections and their settings:

- Carousel: images, aspect ratio
- Text section: title, description, colors
- CTA: label, URL, colors

Storage approach is flexible (e.g. SQLite, Postgres, file-based, or in-memory), but tradeoffs must be documented in the README.

API Design

- Use **REST or GraphQL** (choose one).

- Implement endpoints or operations to:
    - Load a configuration by ID
    - Create a configuration
    - Update an existing configuration

- Apply **best practices for API contracts**, including:
    - Clear, explicit request and response schemas
    - Validation of inputs (e.g. hex colors, URLs)
    - Consistent error responses
    - Appropriate HTTP status codes (for REST) or typed errors (for GraphQL)
    - Versioning or schema evolution considerations

Private Authentication

- The configuration API must be protected using **private authentication**.
- Authentication credentials (e.g. API key, token, secret) must:
    - Live only on the server
    - Never be exposed to the browser
- Direct client-side calls from the browser to the configuration service using credentials are **not allowed**.

Server-Side API Access (Required)

All interaction between the UI and the configuration service must go through **server-side loaders/actions**:

- Use loaders to fetch configuration data on initial page load.
- Use server actions to persist configuration updates.
- The browser should communicate only with your app's server layer, not directly with the configuration service.

Reloading the page must restore the persisted configuration.

Suggested User Flow

1. App loads with a default or previously saved configuration.
2. User edits the home screen using the editor.
3. Preview updates immediately.
4. User saves changes.
5. Refreshing the page restores the saved configuration.

(Autosave or multiple configurations are optional enhancements.)

Technical and Code Quality Requirements

- Clean, modular, maintainable code with clear separation of concerns:

    - UI components
    - State management
    - Server-side data access
    - API contract definitions and validation
- Meaningful automated tests for the core editor/preview behavior.
- Environment variables for secrets and configuration.
- Sensible defaults and graceful error handling.

AI Usage

AI tools are allowed and encouraged. Include a short **Decision Log** (1–2 pages max) covering:

- How you used AI
- What you changed or rejected
- Key technical tradeoffs
- What you would improve with more time

Submission

- Provide a link to a public repository (GitHub, GitLab, etc.) containing the code.
- Include a README file with:
    - Local setup instructions
    - Required environment variables
    - How to run the app (and backend, if separate)
    - High-level architecture overview
    - Notable tradeoffs or assumptions
- Decision Log

Evaluation Criteria

- Functionality: the editor, live preview, import/export, and persistence work as expected.
- API & Architecture: well-designed API contracts, proper server-side mediation, and private authentication.
- Code Quality: readable, well-structured, tested, and maintainable.
- Foundations & Extensibility: clean seams that would be straightforward to extend.
- User Experience: intuitive, responsive, and handles bad input gracefully.
- Pragmatism & Clarity: thoughtful tradeoffs and clear documentation.

Good luck and have fun building! If you have any questions or need clarifications, feel free to reach out.